Audio Filtering Using the Fourier Transform in MATLAB

1st Zekarias Jemberu
Department of Electrical and Computer Engineering
Abu Dhabi University
Abu Dhabi, United Arab Emirates
Email: 1093900@students.adu.ac.ae

2nd Wondu Biru
Department of Electrical and Computer Engineering
Abu Dhabi University
Abu Dhabi, United Arab Emirates
Email: 1093888@students.adu.ac.ae

3rd Mohamed Hamad
Department of Electrical and Computer Engineering
Abu Dhabi University
Abu Dhabi, United Arab Emirates
Email: 1086876@students.adu.ac.ae

***Abstract*—This project presents an audio filtering system implemented in MATLAB that uses the Fourier Transform to perform frequency-domain filtering. The main goal is to remove unwanted high-frequency noise from audio signals while preserving the clarity and essential features of the original sound. By using the Fast Fourier Transform (FFT), the system achieves efficient filtering compared to traditional time-domain convolution. A simple MATLAB GUI was developed to allow users to record or load a six-second audio clip, apply a low-pass FIR filter, visualize both time- and frequency-domain representations, and compare the original and filtered audio through playback. This work provides practical experience with digital signal processing concepts, bridging theory and hands-on experimentation, and demonstrates the effectiveness of frequency-domain filtering for real-world audio applications.**

***Index Terms*—Fourier Transform, Audio Filtering, Frequency Domain, MATLAB, Signal Processing, GUI, FIR Filter**

I. Introduction

Filtering is a core operation in digital signal processing (DSP) that allows the removal of unwanted noise, enhancement of desired signal components, and shaping of a signal's spectral characteristics. Traditionally, filtering is carried out in the time domain by convolving an input signal with a filter's impulse response. While this approach is straightforward conceptually, it becomes computationally intensive when processing long-duration audio signals. The Fourier Transform offers a more efficient alternative by converting convolution operations in the time domain into simple multiplications in the frequency domain [1]. This principle forms the foundation of frequency-domain filtering and enables faster processing, especially for audio and other high-sample-rate signals.

The primary objective of this project is to develop an audio filtering system in MATLAB that leverages the Fourier Transform for frequency-domain filtering. A low-pass Finite Impulse Response (FIR) filter is specifically implemented to suppress high-frequency noise while maintaining the integrity and clarity of the original audio signal. This project not only reinforces theoretical understanding of the Discrete Fourier Transform (DFT) and its efficient computation via the Fast Fourier Transform (FFT) but also demonstrates how these concepts are applied in practical audio signal processing [2].

To make the system interactive and user-friendly, a MATLAB Graphical User Interface (GUI) was created using the `uifigure`, `uibutton`, and `uiaxes` functions [3]. Through the GUI, users can load or record a six-second audio clip, apply the low-pass filter, visualize both time- and frequency-domain representations, and compare the original and filtered audio via playback. This hands-on approach bridges theoretical DSP concepts with practical experimentation, giving students and users valuable experience in filter design, spectral analysis, and interactive system development in MATLAB [4].

II. Theory of Frequency-Domain Filtering

The theoretical basis for frequency-domain filtering relies on the Convolution Theorem, which states that convolution in the time domain corresponds to multiplication in the frequency domain. For a continuous-time system, this relationship is expressed as:

$$y(t) = x(t) * h(t), \tag{1}$$

where $x(t)$ is the input signal and $h(t)$ is the filter's impulse response. Transforming this relationship to the frequency domain via the Fourier Transform yields:

$$Y(f) = X(f) \cdot H(f), \tag{2}$$

where $X(f)$ and $H(f)$ are the Fourier Transforms of the input signal and the filter, respectively [1]. This equivalence demonstrates that filtering can be achieved by multiplying the signal and filter spectra, eliminating the need for computationally expensive time-domain convolution.

This frequency-domain approach offers significant computational advantages. While direct convolution has a complexity of $O(N^2)$, frequency-domain filtering using the FFT reduces this to $O(N \log N)$, making it highly efficient for processing long audio signals [2]. Once the spectral multiplication is

performed, the filtered signal is reconstructed in the time domain using the Inverse Fast Fourier Transform (IFFT) [3].

In this project, a low-pass FIR filter was designed using MATLAB's `fir1()` function [5]. The filter suppresses frequencies above a chosen cutoff while preserving the primary audio content. Its coefficients are calculated as:

$$h[n] = \text{fir1}(N, f_c/(f_s/2)), \tag{3}$$

where N is the filter order, f_c is the cutoff frequency, and f_s is the sampling rate. For this implementation, a cutoff frequency of 1000 Hz was chosen, effectively removing high-frequency noise from the input voice or music signal [4]. This theoretical framework underpins the system's subsequent design, implementation, and evaluation.

III. System Design and Implementation

A. Overview

The audio filtering system is designed to process a six-second segment of a user-provided or recorded audio signal, apply a low-pass FIR filter in the frequency domain, and provide visual and auditory feedback through a MATLAB GUI [1]. The workflow begins with loading an existing .wav file or recording a new voice sample, ensuring that only the first six seconds are processed and analyzed. The system then designs a low-pass FIR filter with a predetermined cutoff frequency [2], computes the FFTs of both the input signal and the filter [3], and multiplies their spectra to perform frequency-domain filtering [1], [4]. The filtered signal is recovered using the inverse FFT, displayed alongside the original for comparison, and saved as a new audio file [3]. All of these steps are automated within the MATLAB GUI, allowing users to perform the entire filtering operation with minimal manual intervention.

B. Graphical User Interface (GUI)

The MATLAB GUI provides an intuitive and interactive interface for user operation [1]. It includes buttons for loading audio, recording new samples, applying the filter, and playing back both the original and filtered audio [2]. The GUI also features six axes to visualize the input signal, the filter characteristics, and the filtered output in both time and frequency domains. Figure 1 shows a schematic layout of the GUI, demonstrating the arrangement of controls and plotting areas. This layout allows users to easily interact with the system and observe the filtering effects in real time.

Fig. 1: MATLAB GUI layout for the audio filtering system [1].

C. Signal Processing Workflow

When a user loads or records an audio sample, the system first truncates it to the initial six seconds for analysis. Figure 2 shows the input audio signal in the time domain. This plot provides a visual understanding of the raw waveform before any filtering, highlighting the amplitude variations and transient components that the filter will act upon.

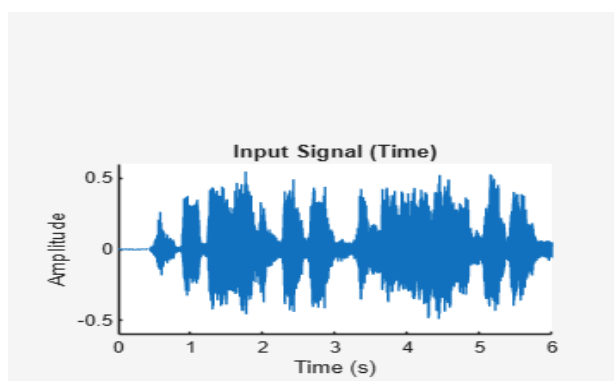

Fig. 2: Input signal in the time domain [3].

Next, the system applies a low-pass FIR filter. Figure 3 illustrates the impulse response of the designed FIR filter. The shape and duration of the impulse response determine how the filter modifies the input signal, attenuating frequencies above the cutoff while preserving the desired low-frequency content.

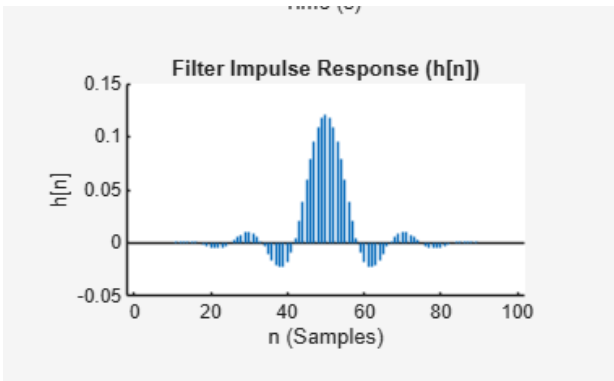

Fig. 3: Impulse response of the FIR low-pass filter [2].

After applying the filter in the frequency domain, the reconstructed time-domain signal is shown in Figure 4. Comparing this with the original waveform, it is evident that high-frequency noise has been reduced while the main audio content is preserved [3].

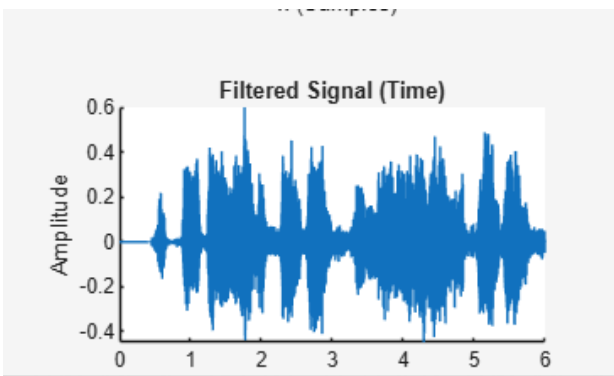

Fig. 4: Filtered signal in the time domain [3].

D. Frequency-Domain Analysis

Visualizing signals in the frequency domain provides further insight into the filtering process. Figure 5 shows the magnitude spectrum of the input signal. Peaks at higher frequencies indicate the noise components targeted by the low-pass filter [3].

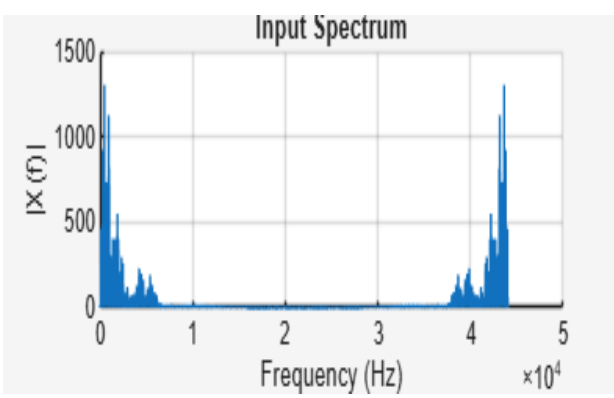

Fig. 5: Magnitude spectrum of the input signal [3].

Figure 6 depicts the magnitude spectrum of the FIR filter. The flat response in the passband and sharp attenuation beyond the cutoff frequency illustrate the filter's effectiveness [2].

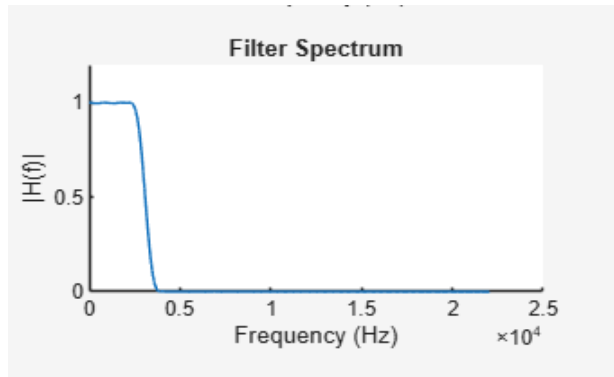

Fig. 6: Magnitude spectrum of the filter [2].

Finally, the magnitude spectrum of the filtered signal is shown in Figure 7. The high-frequency components are significantly reduced compared to the original, confirming the successful application of the low-pass filtering in the frequency domain [3].

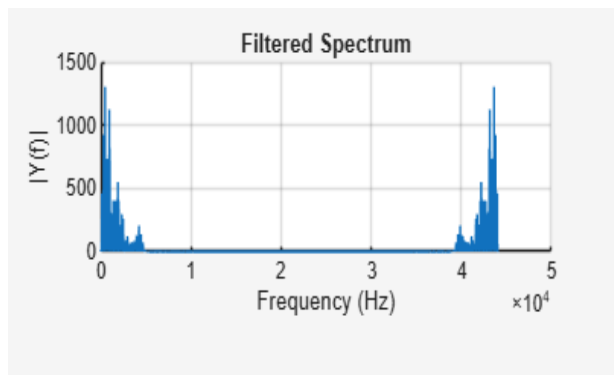

Fig. 7: Magnitude spectrum of the filtered signal [3].

The MATLAB GUI and signal processing workflow described above are fully implemented in the MATLAB script provided in Appendix 1. This code handles audio loading, recording, FIR filter design, frequency-domain filtering, and real-time plotting of signals in both time and frequency domains.

IV. Results and Discussion

The audio filtering system developed in this project was tested using a six-second voice recording sampled at 44.1 kHz. The main objective was to remove unwanted high-frequency noise while preserving the natural timbre and clarity of the original speech. A low-pass FIR filter with a cutoff frequency of 1000 Hz was applied, effectively reducing background hiss and sharp noise components without affecting the primary voice frequencies.

Frequency-domain filtering using the FFT was employed to perform this task efficiently. By transforming both the audio signal and the filter into the frequency domain, filtering was achieved through simple spectral multiplication, avoiding computationally expensive time-domain convolution. The filtered signal was then reconstructed in the time domain using the inverse FFT, producing a smooth waveform with reduced high-frequency oscillations.

Figures 2–7 illustrate the filtering process. The original time-domain waveform (Figure 2) exhibits sharp transients and high-frequency components, while the filtered waveform (Figure 4) appears smoother and more stable. The magnitude spectra show a clear suppression of high-frequency components above the cutoff frequency (Figures 5 and 7). Listening to the filtered audio confirmed these observations: the sound was cleaner and softer, with diminished hissing and no noticeable distortion, demonstrating the effectiveness of the FFT-based approach.

The MATLAB GUI provided an interactive platform to visualize each step of the process. Users could observe the input signal, the filter's impulse response, and the resulting filtered signal in both time and frequency domains. Playback buttons allowed comparison of the original and filtered audio, reinforcing understanding of how the FIR filter shapes the signal and how frequency-domain filtering operates in practice.

Through this project, we gained practical experience in FIR filter design, FFT-based signal processing, and GUI development in MATLAB. Users can clearly see the effects of filtering and experiment with different parameters. Future improvements could include adding a real-time adjustable cutoff frequency slider, implementing other filter types such as high-pass or band-pass, and providing live spectrum visualization during playback. These enhancements would make the system more versatile, educational, and engaging for hands-on learning.

V. Conclusion

This project successfully implemented a MATLAB-based audio filtering system that leverages frequency-domain processing to efficiently remove high-frequency noise. By applying a low-pass FIR filter via spectral multiplication, the system preserved the clarity and natural quality of the original audio while reducing unwanted artifacts. The interactive GUI enabled users to load or record audio, apply the filter, visualize both time- and frequency-domain representations, and listen to the results, providing a comprehensive and intuitive experience.

The project provided valuable learning opportunities, including understanding the FFT, FIR filter design, spectral multiplication, and interactive application development. It bridged theoretical digital signal processing concepts with practical implementation, showing how DSP techniques can be applied to real-world audio processing tasks. Incorporating features like adjustable filter parameters and real-time visualization could further enhance the system's educational and practical value.

Acknowledgment

The authors would like to express their sincere gratitude to Dr. Hatem for his guidance, and support throughout the CEN320 course project. His supervision was invaluable in completing the project successfully and in gaining practical insights into digital signal processing and MATLAB-based system design.

Appendix A
MATLAB Code: Audio Filtering GUI

Listing 1: Complete MATLAB script for the Audio Filtering GUI using FFT and FIR filter.

```
function AudioFilterGUI
    % ===== MAIN FIGURE =====
    fig = uifigure('Name','Audio Filter GUI','
        Position',[200 100 820 680]);
    movegui(fig,'center');

    % Initialize data storage
    data.audio = []; data.fs = 44100; data.filtered
        = []; data.cutoff = 1000;
    guidata(fig,data);

    % ===== BUTTONS =====
    uibutton(fig,'Text','Load Audio (.wav)','
        Position',[40 620 150 30],'ButtonPushedFcn',
        @(~,~)loadAudio(fig));
    uibutton(fig,'Text','Record Audio (5 s)','
        Position',[210 620 150 30],'ButtonPushedFcn'
        ,@(~,~)recordAudio(fig));
    uibutton(fig,'Text','Filter & Plot','Position'
        ,[380 620 120 30],'ButtonPushedFcn',@(~,~)
        filterAndPlot(fig));
    uibutton(fig,'Text','Play Original','Position'
        ,[520 620 120 30],'ButtonPushedFcn',@(~,~)
        playOriginal(fig));
    uibutton(fig,'Text','Play Filtered','Position'
        ,[660 620 120 30],'ButtonPushedFcn',@(~,~)
        playFiltered(fig));

    % ===== CUTOFF SLIDER =====
    uilabel(fig,'Text','Cutoff (Hz):','Position',[40
         580 80 20]);
    uislider(fig,'Position',[120 590 220 3],'Limits'
        ,[100 10000],'Value',1000,'ValueChangedFcn',
        @(sld,~)adjustCutoff(fig,round(sld.Value)));

    % ===== PLOT AREAS =====
    for i=1:6
        ax(i) = uiaxes(fig,'Position',[40+mod(i-1,2)
            *370,380-floor((i-1)/2)*160,350,150]);
        grid(ax(i),'on');
    end
    setappdata(fig,'axes',ax);
end

%% ===== CALLBACK FUNCTIONS =====

function loadAudio(fig)
    [file,path] = uigetfile('*.wav','Select audio
        file');
    if isequal(file,0), uialert(fig,'No file
        selected.','Warning'); return; end
    [y,fs] = audioread(fullfile(path,file));
    if size(y,2) > 1, y = mean(y,2); end
    y = y(1:min(end,6*fs)); % Limit to first 6
        seconds
    data = guidata(fig); data.audio = y; data.fs =
        fs; guidata(fig,data);
```

```
    uialert(fig,'Audio loaded successfully!','Success');
end

function recordAudio(fig)
    fs = 44100; recObj = audiorecorder(fs,16,1);
    uialert(fig,'Recording 5 seconds...','Recording'); recordblocking(recObj,5);
    y = getaudiodata(recObj);
    data = guidata(fig); data.audio = y; data.fs = fs; guidata(fig,data);
    uialert(fig,'Recording complete!','Done');
end

function filterAndPlot(fig)
    data = guidata(fig); ax = getappdata(fig,'axes');
    if isempty(data.audio), uialert(fig,'Load or record audio first.','Error'); return; end

    x = data.audio; fs = data.fs; fc = data.cutoff;
    M = 101; % FIR filter length
    h = fir1(M-1, fc/(fs/2)); % Design low-pass FIR filter

    % ----- APPLY FILTER -----
    y = conv(x,h,'same'); % Linear convolution
    data.filtered = y; guidata(fig,data);
    audiowrite('filtered_output.wav',y,fs);

    % ===== PLOTS =====
    t = (0:length(x)-1)/fs;
    L_fft = length(y) + M -1;
    f = (0:L_fft-1)*(fs/L_fft);
    X_fft = fft(x,M+length(x)-1); Y_fft = fft(y,M+length(x)-1); H_fft = fft(h,M+length(x)-1);

    plot(ax(1),t,x); title(ax(1),'Input Signal (Time)'); xlabel(ax(1),'Time (s)'); ylabel(ax(1),'Amplitude');
    plot(ax(2),h); title(ax(2),'Filter Impulse Response'); xlabel(ax(2),'Samples'); ylabel(ax(2),'h[n]');
    plot(ax(3),t,y); title(ax(3),'Filtered Signal (Time)'); xlabel(ax(3),'Time (s)'); ylabel(ax(3),'Amplitude');
    plot(ax(4),f(1:length(x)),abs(X_fft(1:length(x))); title(ax(4),'Input Spectrum'); xlabel(ax(4),'Frequency (Hz)'); ylabel(ax(4),'|X(f)|');
    plot(ax(5),f(1:length(h)),abs(H_fft(1:length(h))); title(ax(5),'Filter Spectrum'); xlabel(ax(5),'Frequency (Hz)'); ylabel(ax(5),'|H(f)|');
    plot(ax(6),f(1:length(y)),abs(Y_fft(1:length(y))); title(ax(6),'Filtered Spectrum'); xlabel(ax(6),'Frequency (Hz)'); ylabel(ax(6),'|Y(f)|');
end

function playOriginal(fig)
    data = guidata(fig);
    if isempty(data.audio), uialert(fig,'Load or record audio first.','Warning'); return; end
    sound(data.audio,data.fs);
end

function playFiltered(fig)
    data = guidata(fig);
    if isempty(data.filtered), uialert(fig,'Press "Filter & Plot" first.','Warning'); return; end
    sound(data.filtered,data.fs);
end

function adjustCutoff(fig,newValue)
    data = guidata(fig); data.cutoff = newValue; guidata(fig,data);
    fprintf('Cutoff frequency updated to %d Hz\n', newValue);
end
```

References

[1] C.-L. Lee, J.-H. Park, and S.-W. Kim, "Enhanced Adaptive Wiener Filtering for Frequency-Varying Noise with Convolutional Neural Network-Based Feature Extraction," *Sensors (MDPI)*, vol. 25, no. 3, pp. 1001–1015, Feb. 2025.

[2] T. Tarafder, R. Rahman, and M. Ali, "A Comprehensive Review of Digital Signal Processing (DSP) Algorithms and Their Applications in Telecommunication and Wireless Communication Systems," *Int. J. Eng. Technol. Sci.*, vol. 15, no. 1, pp. 50–65, Jan. 2025.

[3] V. Thomas, S. Nair, and R. Kumar, "Real Time Noise Reduction Using Machine Learning with the Filtering Techniques," *Int. J. Res. Pub. Revs.*, vol. 4, no. 6, pp. 1120–1125, Jun. 2023.

[4] J. Urena, A. Hernandez, P. Martin, and R. Garcia, "Acoustic Local Positioning With Encoded Emission Beacons," *ArXiv:2401.00000 [eess.SP]*, Jan. 2024. [Online]. Available: http://arxiv.org/abs/2401.00000

[5] MathWorks, Inc., "fir1: FIR filter design using window method," *Signal Processing Toolbox Documentation*, 2024. [Online]. Available: https://www.mathworks.com/help/signal/ref/fir1.html